



المعهد الوطني للبريد والمواصلات
ⵎⵓⵙⵙⵓⵏⵉⵎ ⵙⵉⵎⵓⵙⵓⵏⵉⵎ ⵙⵉⵎⵓⵙⵓⵏⵉⵎ ⵙⵉⵎⵓⵙⵓⵏⵉⵎ
Institut National des Postes et Télécommunications

Transformers

Attention Mechanisms

Pr. Tarik Fissaa

ICCN – INE2

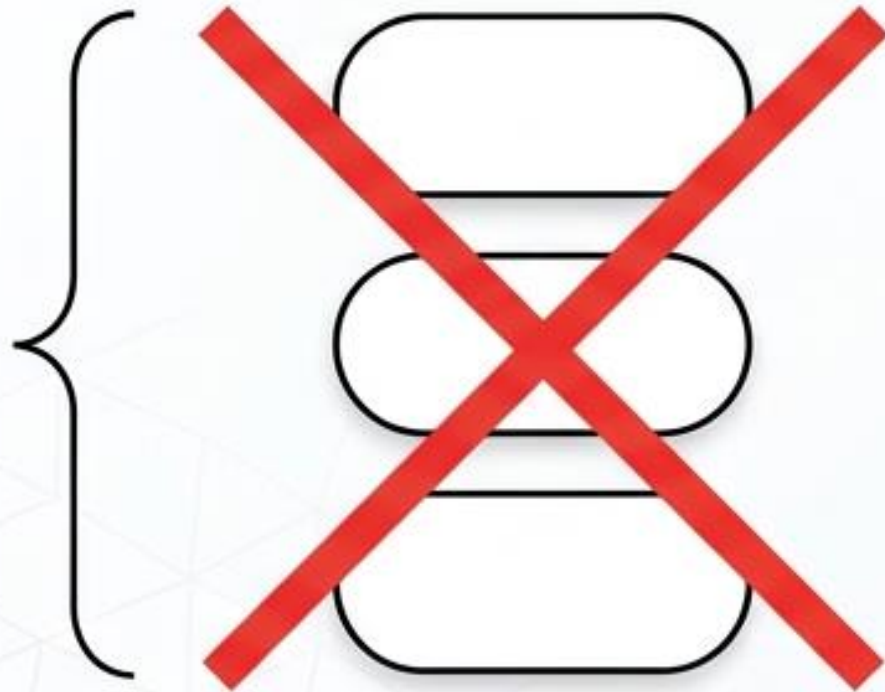
Attention Is All You Need

Decoding the Mathematical Engine of the
Transformer Architecture.

The Sequential Bottleneck of Recurrent Models

The Constraint:

Recurrent models factor computation along symbol positions.
Generating state h_t requires the completion of h_{t-1} .



The Consequence:

This inherently sequential nature precludes parallelization within training examples.

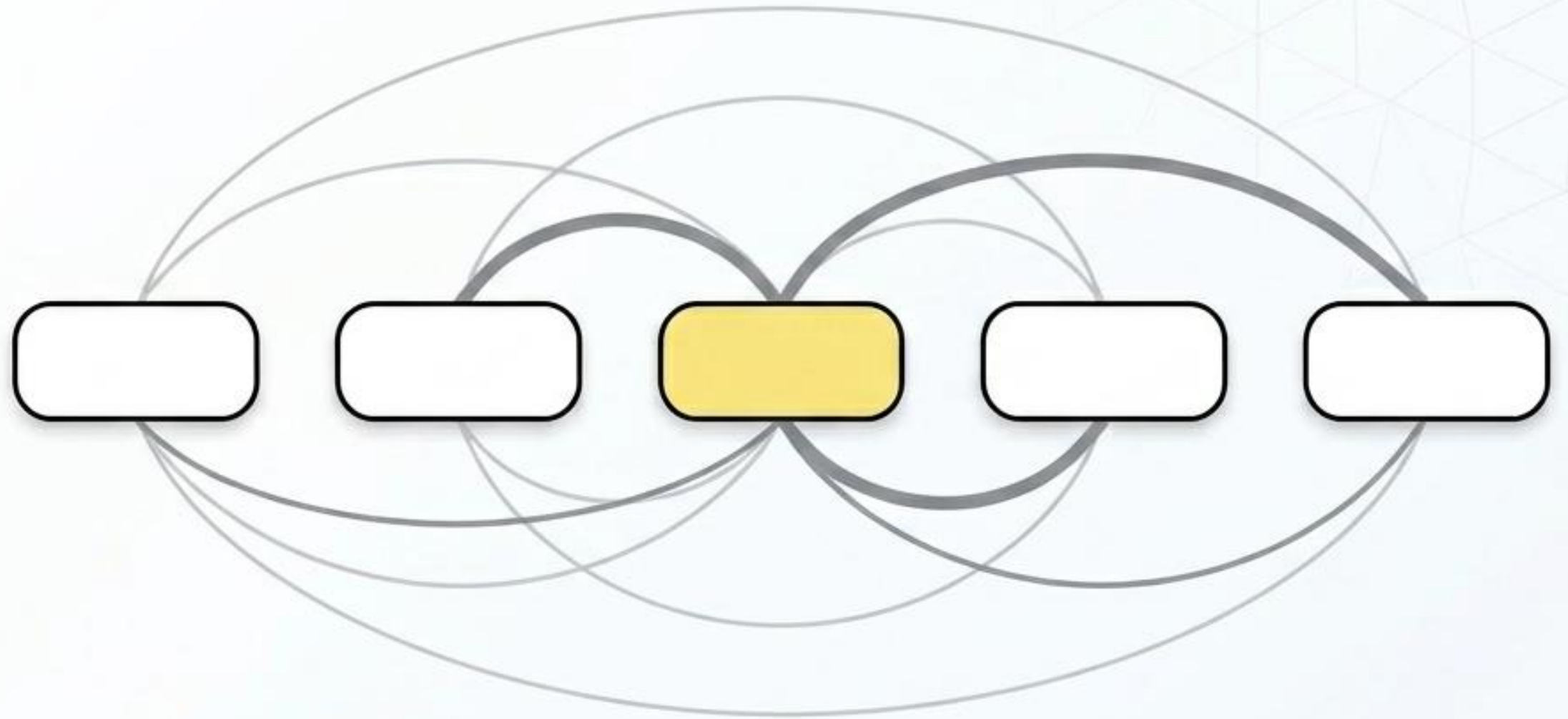
The Limit:

As sequence lengths grow, memory constraints severely limit batching across examples.

Dropping Recurrence for Global Dependencies

The Transformer architecture eschews recurrence entirely.

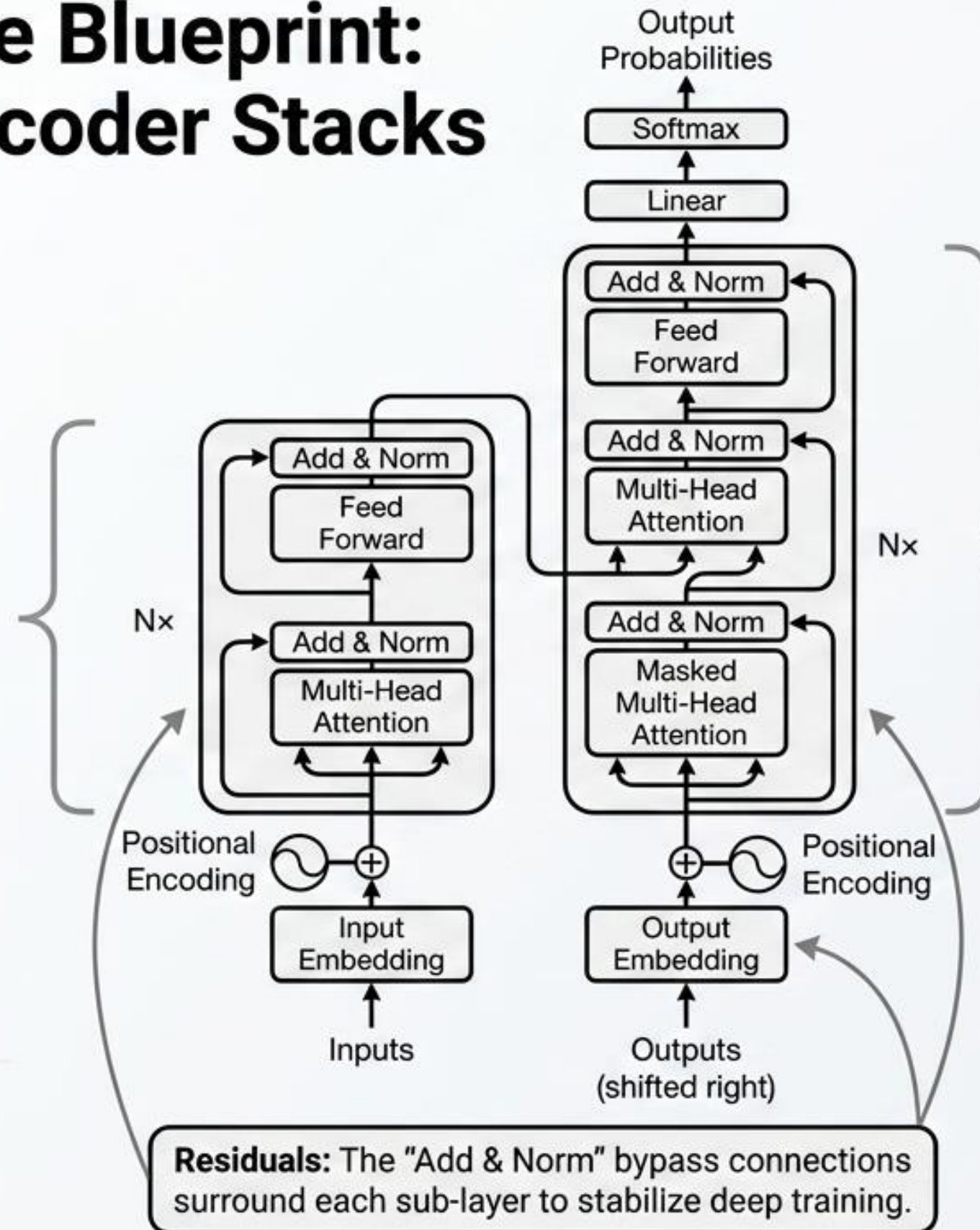
It relies solely on an attention mechanism to draw global dependencies between input and output.



Impact: Reduces the number of sequential operations to $O(1)$, unlocking massive parallelization and state-of-the-art translation quality in a fraction of the training time.

The Architecture Blueprint: Encoder and Decoder Stacks

Encoder: Maps input sequence to continuous representations. Uses **Multi-Head Self-Attention**.



Decoder: Auto-regressive. Consumes previously generated symbols. Includes Masked Attention.

The Metaphor: Queries, Keys, and Values

Q



What you are looking for.

The active representation of the current word trying to understand its context.

K



What I contain.

The label that each word in the sequence broadcasts to the rest of the sentence.

V

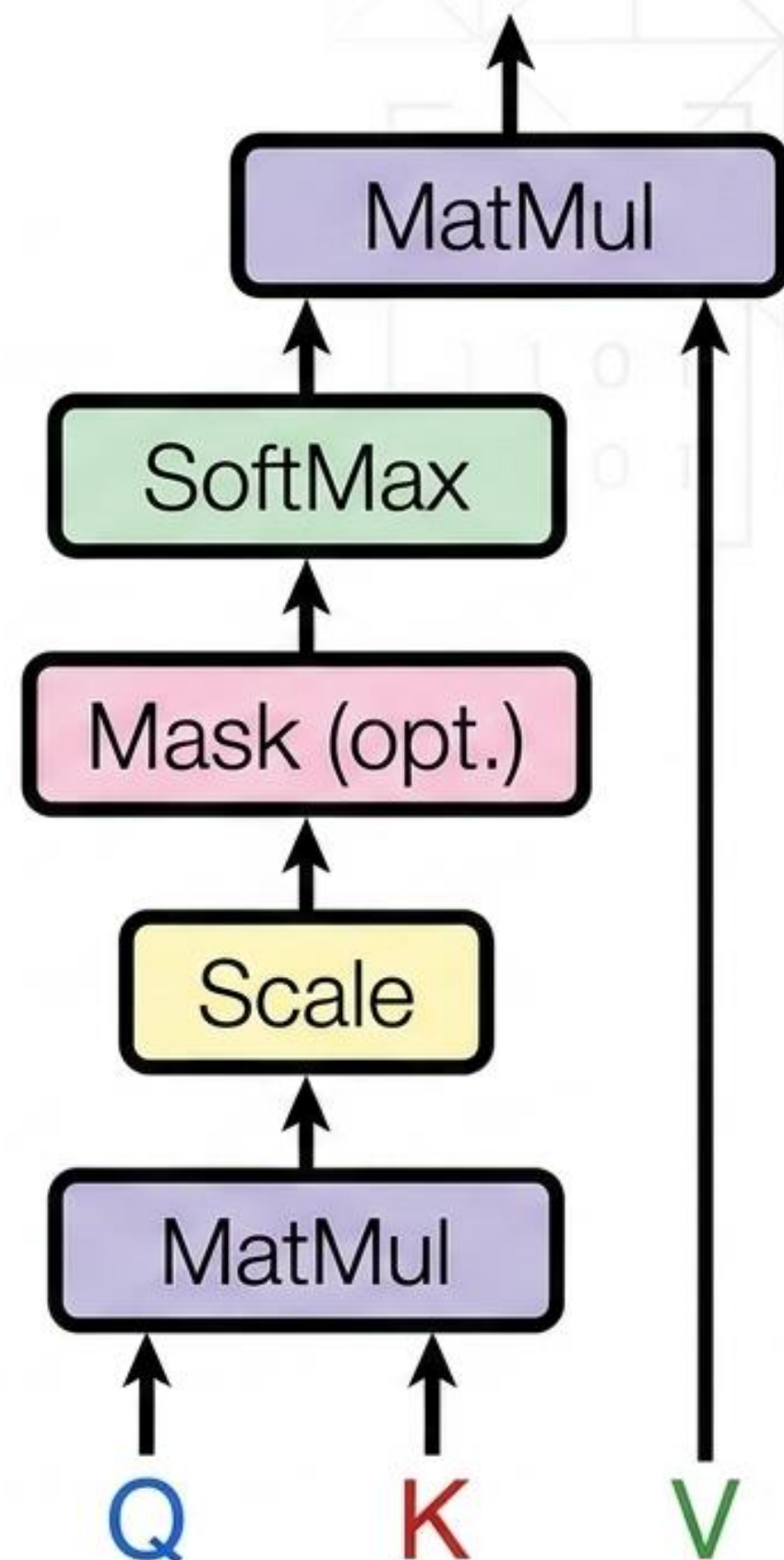
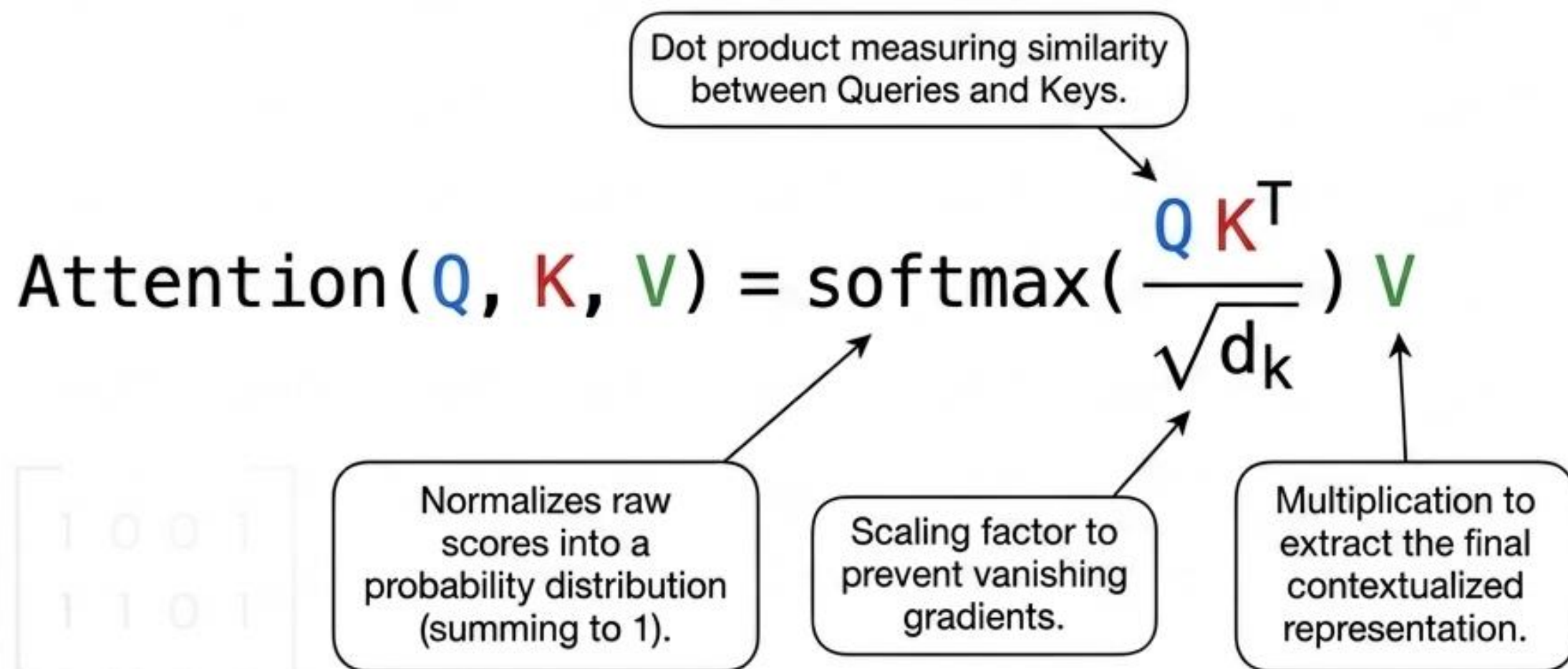


The actual content.

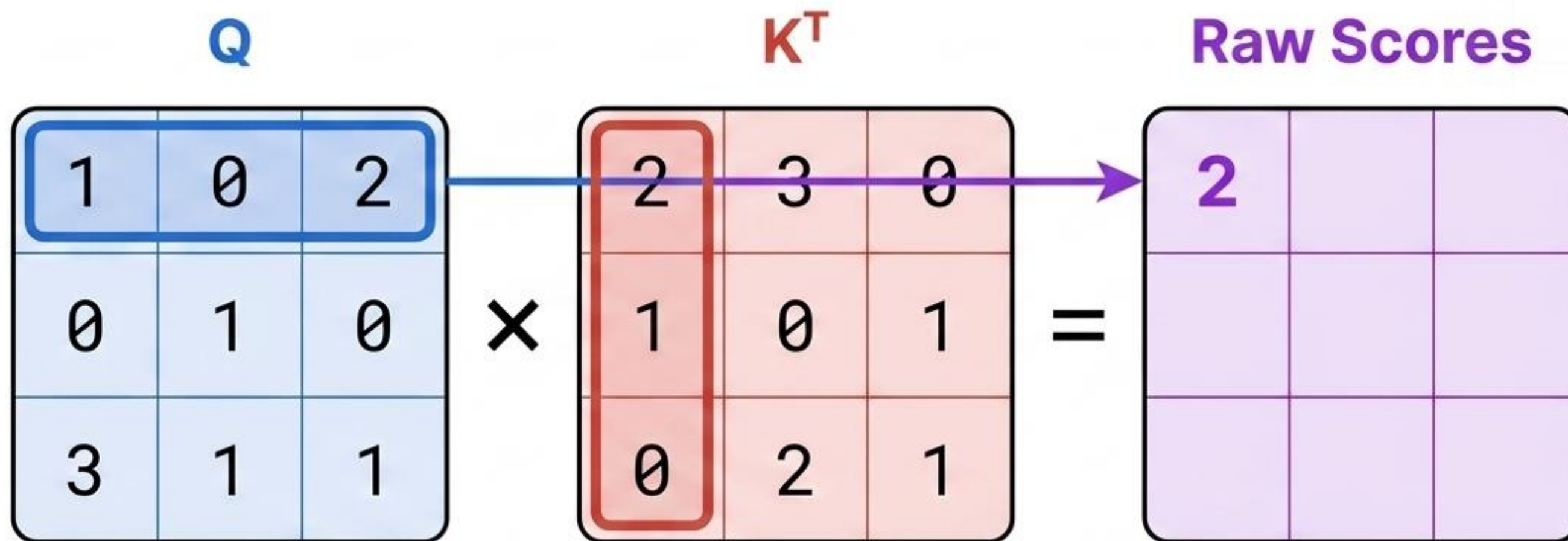
The underlying meaning of the word that will be extracted if the Query matches the Key.

Attention is computed as a weighted sum of the Values (Green), where the weight is calculated by how well the Query (Blue) matches the Key (Red).

The Master Equation: Scaled Dot-Product Attention



Math Walkthrough Step 1: The Match ($Q \times K^T$)

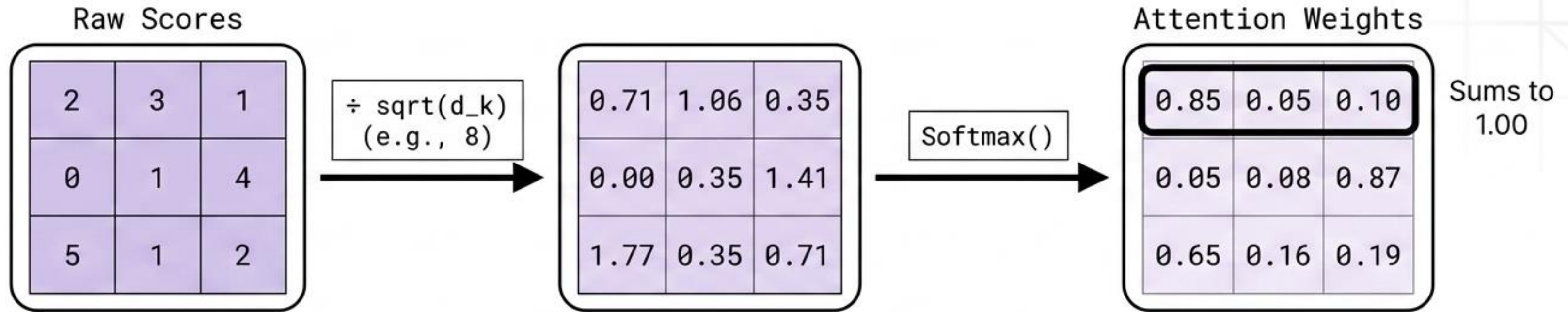


Calculation:

$$(1 \times 2) + (0 \times 1) + (2 \times 0) = 2$$

Meaning: The dot product determines how strongly Word 1's Query aligns with Word 1's Key. Larger numbers mean higher relevance.

Math Walkthrough Step 2: Scale and Softmax



Why Scale?

For large dimensions, dot products grow massive, pushing the softmax function into regions with extremely small gradients. We divide by $\text{sqrt}(d_k)$ to stabilize learning.

Why Softmax?

It converts raw, unbounded match scores into clean probability distributions.

Math Walkthrough Step 3: Contextualization (x V)

Attention Weights

0.85	0.05	0.10
0.05	0.08	0.87
0.65	0.16	0.19

×

V

1.2	2.3	0.9
0.5	1.8	2.1
1.0	0.7	1.5

=

Contextualized Output

1.13	2.09	1.09
0.61	0.98	1.58
0.88	1.27	1.13

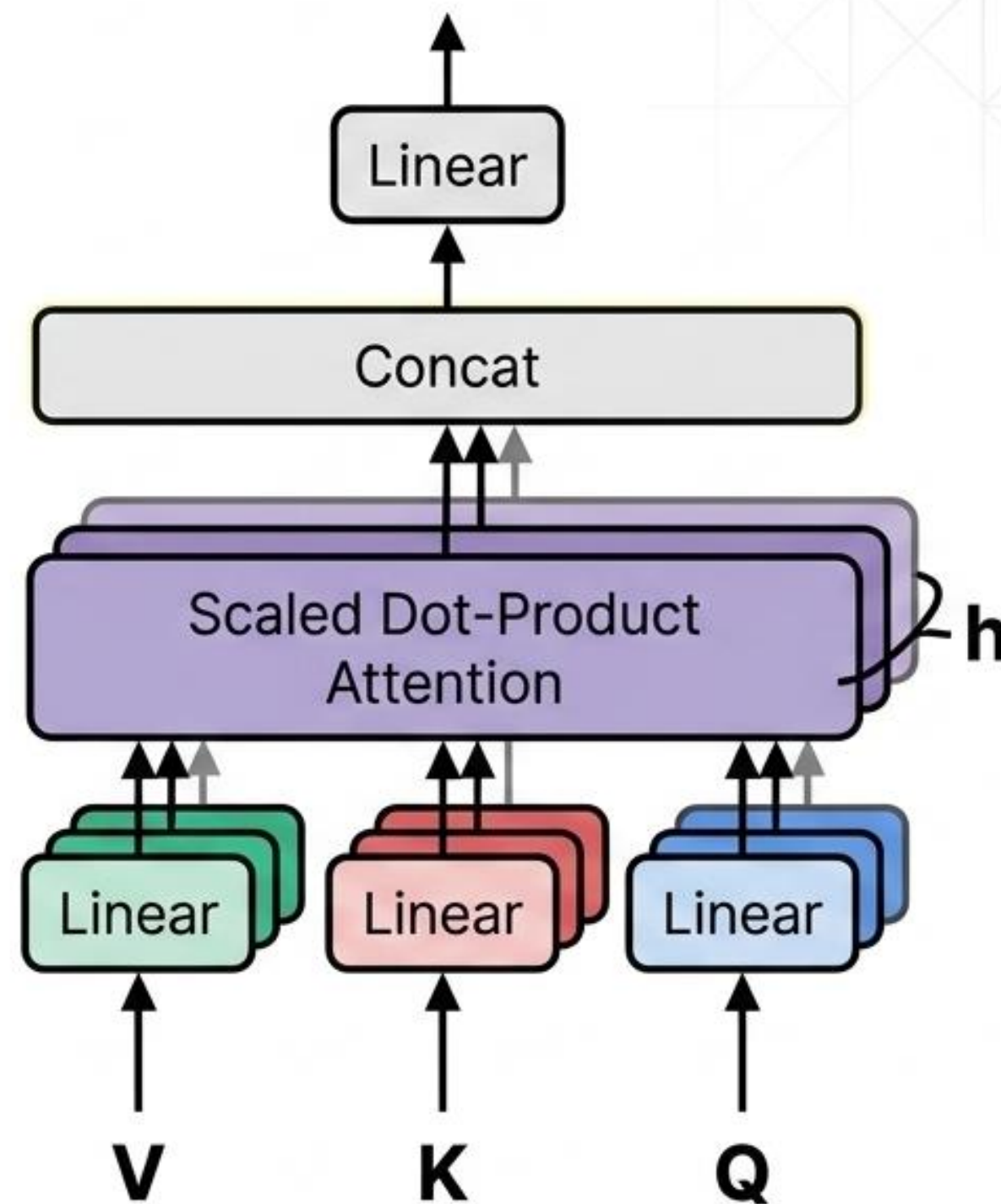
The Insight:

The output for Word 1 is overwhelmingly composed of its own Value (85%), with a slight blend of Word 3's Value (10%) and Word 2's Value (5%). The word is now mathematically contextualized by its neighbors.

Multi-Head Attention: Information from Subspaces

Total dimension	$d_{\text{model}} = 512$
Heads	$h = 8$
Split dimensions	$d_k = d_v = 512 / 8 = 64$

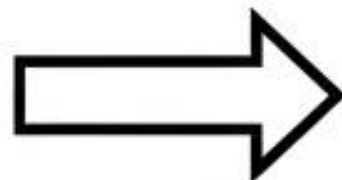
- By linearly projecting the queries, keys, and values h times, the model attends to information from different representation subspaces simultaneously.
- Because the dimension of each head is reduced (64 instead of 512), the total computational cost remains similar to single-head attention with full dimensionality.



Preserving Auto-Regression with Masking

0.85	-inf	-inf	-inf	-inf
0.05	-1.23	-inf	-inf	-inf
1.12	0.10	0.45	-inf	-inf
0.08	2.01	0.98	-0.34	-inf
0.16	0.19	1.54	0.55	0.21

Softmax()

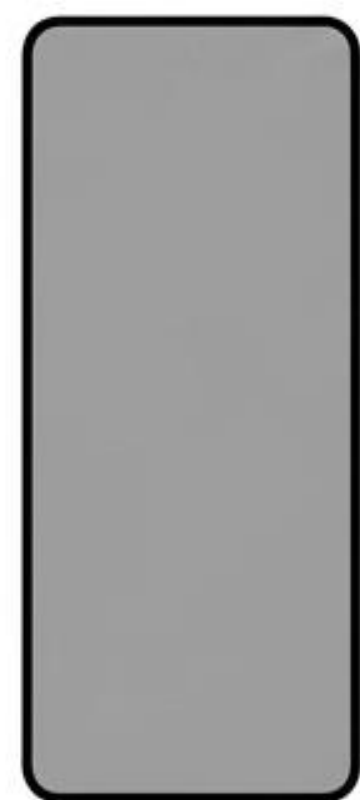


0.54	0.00	0.00	0.00	0.00
0.05	0.01	0.10	0.00	0.00
0.30	0.06	0.06	0.02	0.00
0.80	0.10	0.10	0.02	0.00
0.10	0.12	0.70	0.07	0.01

- In the Decoder, the model must predict the sequence one element at a time. It cannot be allowed to 'look into the future'.
- By masking out illegal rightward connections with -inf before the Softmax step, those positions become zero probabilities. Leftward information flow is preserved.

Injecting Sequence Order: Positional Encoding

$$\begin{aligned} PE_{(\text{pos}, 2i)} &= \sin(\text{pos} / 10000^{2i/d_{\text{model}}}) \\ PE_{(\text{pos}, 2i+1)} &= \cos(\text{pos} / 10000^{2i/d_{\text{model}}}) \end{aligned}$$



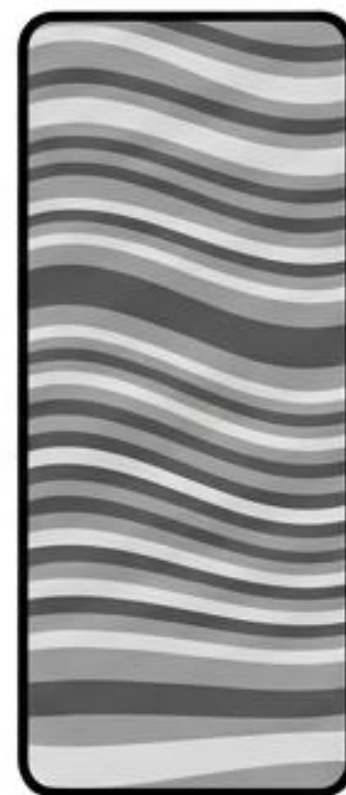
Input
Embedding

+



Positional
Encoding

=



Time-Stamped
Embedding

Since there is no recurrence or convolution, order is injected by adding sine and cosine functions of different frequencies directly to the input vectors at the bottom of the stack.

The Dawn of the Foundation Model Era



Dispensed with Recurrence

Proved that **attention mechanisms alone** are **sufficient** for sequence transduction.



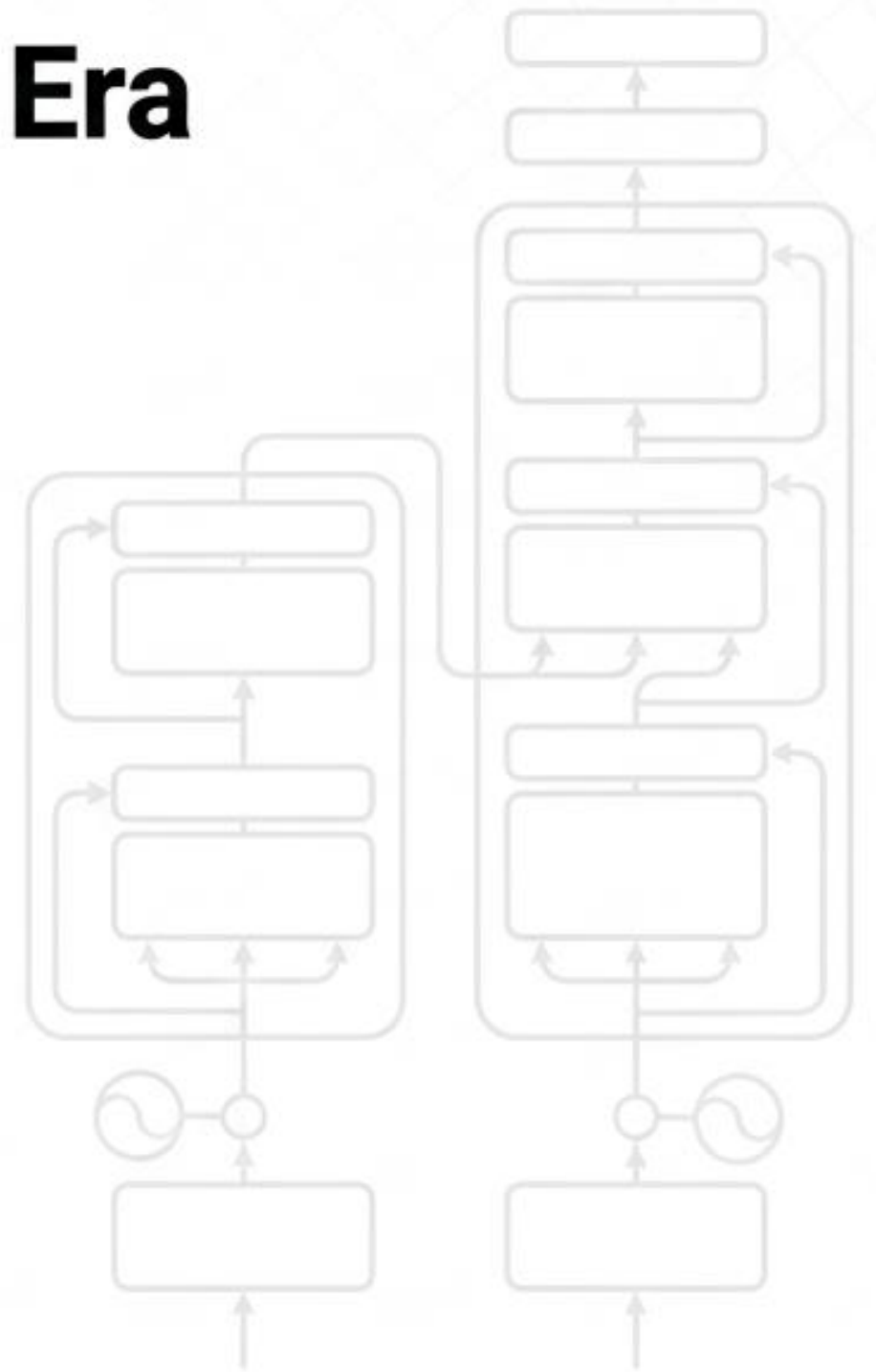
Unlocked Parallelization

Replaced $O(n)$ **sequential bottlenecks** with **constant $O(1)$ operations**, making massive-scale training computationally feasible.



Global Dependency Mapping

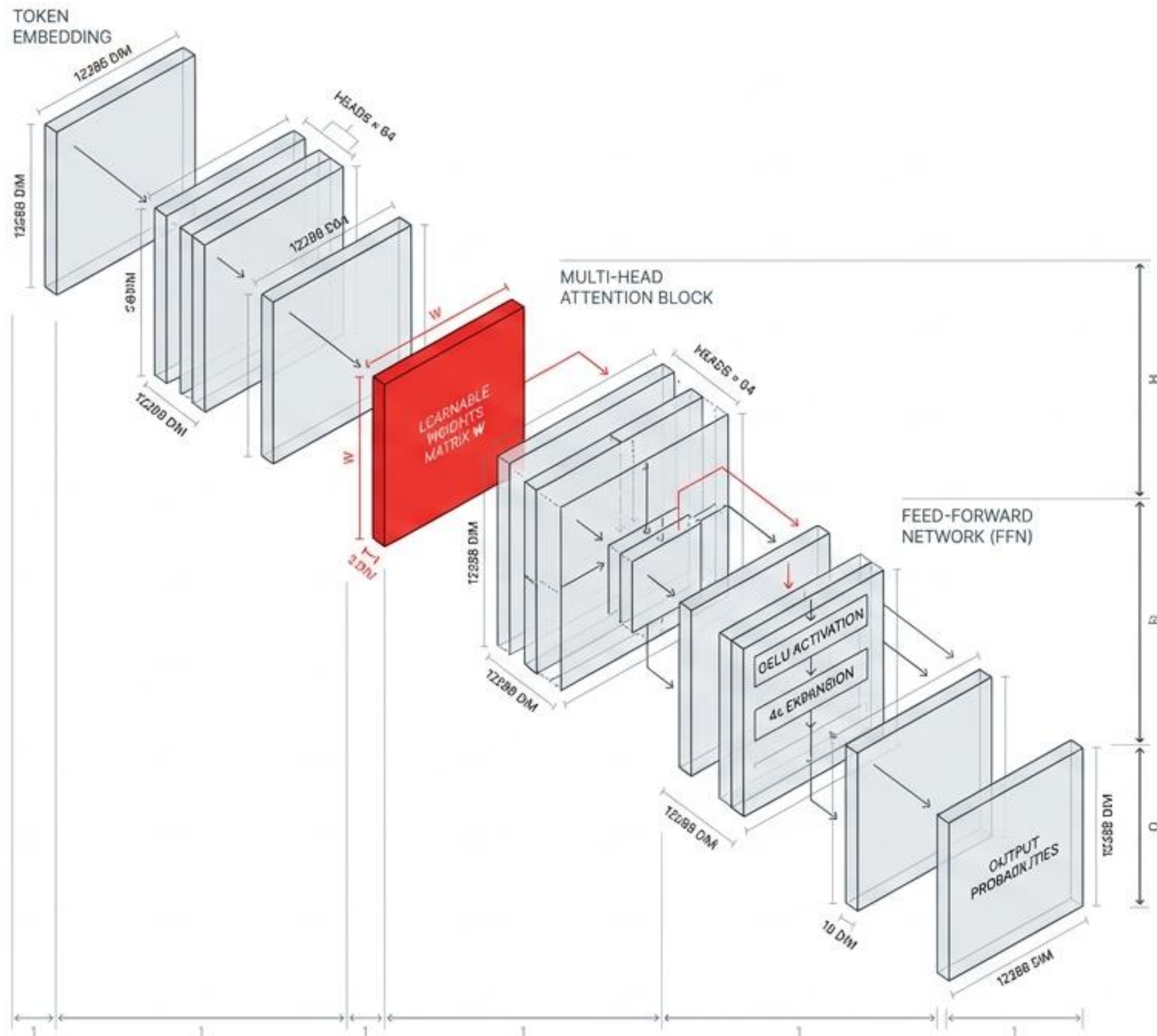
Enabled models to natively understand the **relationship between distant tokens** in a sequence.



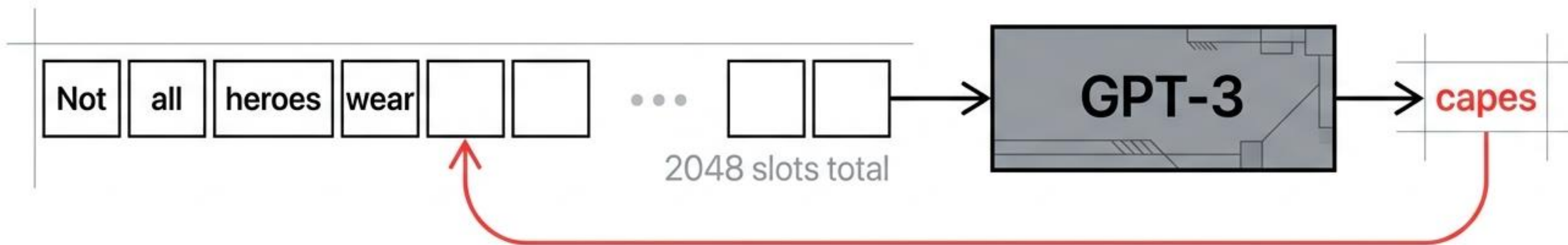
“The Transformer fundamentally shifted sequence modeling from sequential reading to holistic observation.”

Deconstructing the GPT-3 Architecture

A visual guide to the machinery behind a 175-billion parameter language model—mapping the exact mathematical journey from inputs and attention to 96 layers of matrices.



The core mechanic is a simple autoregressive loop.



Input:

A fixed sequence of exactly **2048 tokens**. (Shorter inputs simply fill extra positions with empty values).

Output:

A probability distribution for the single most likely **next** word.

Generation:

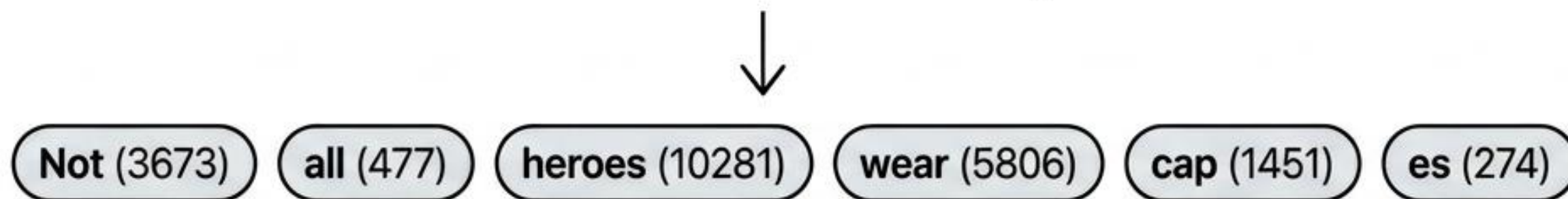
To generate long text, the **output** word is appended to the **input sequence**, and the **cycle repeats**.

Translating words into machine language.

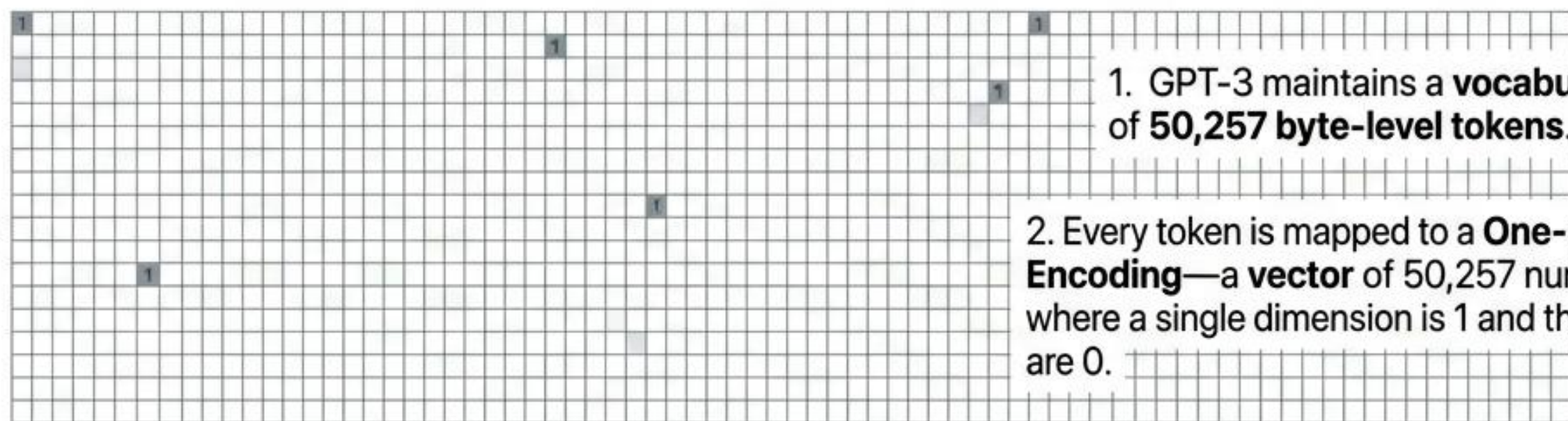
Dimensional
Tracker

Current Matrix
Shape:
2048 x 50257

Not all heroes wear capes



↑
2048
(Sequence
Length)



1. GPT-3 maintains a **vocabulary** of **50,257 byte-level tokens**.

2. Every token is mapped to a **One-Hot Encoding**—a **vector** of 50,257 numbers where a single dimension is 1 and the rest are 0.

50,257
(Vocabulary Size)

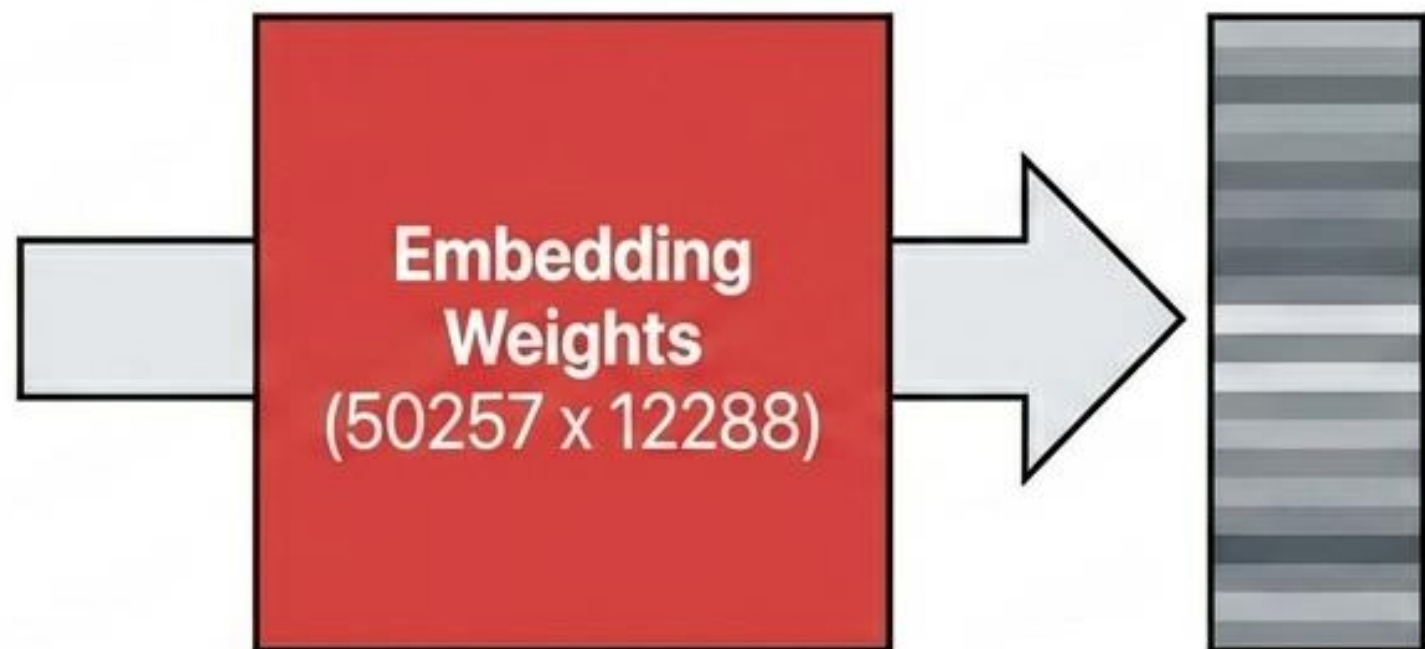
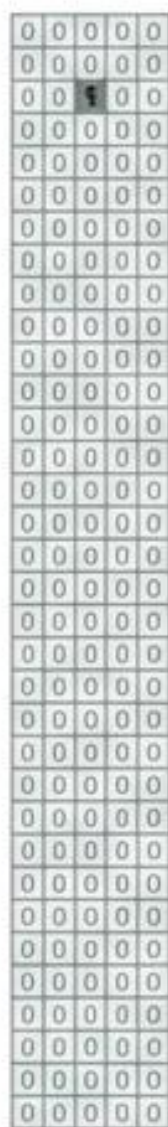


Compressing sparse text into dense semantic meaning.

Dimensional
Tracker

Shape
Compressed:
2048 x 12288

Sparse
One-Hot
Vector
(50,257)



Dense
Embedding
Vector
(12,288)

A 50k-length vector is mostly **wasted space**.
The model learns a projection to map each word to a
coordinate in a **12,288**-dimensional space.

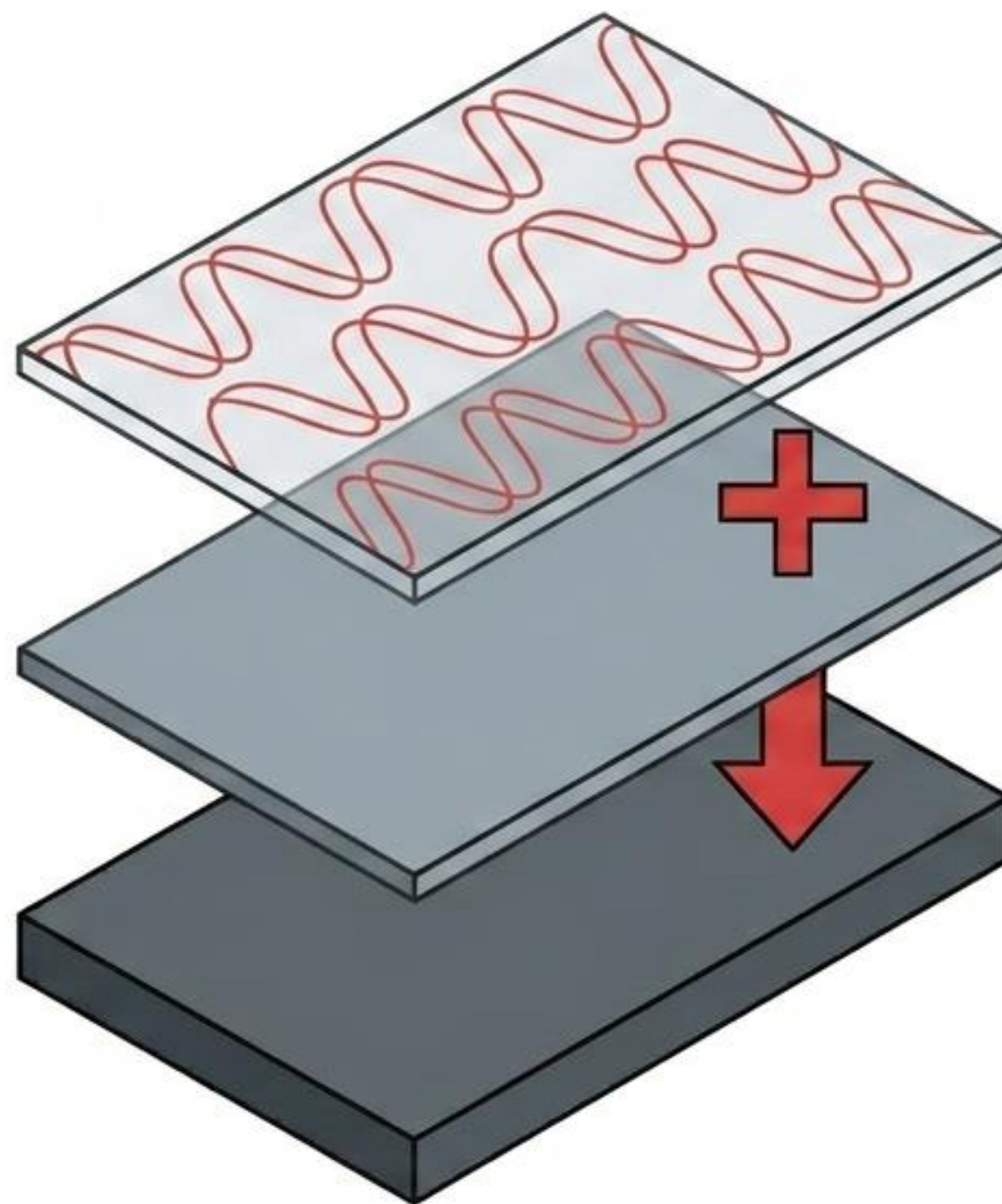
Note: The embedding matrix is applied to each word individually.
At this stage, no information flows across the sequence.

Injecting a sense of time and sequence.

Because embedding happens per word, the model inherently cannot distinguish between **Dog bites man** and **Man bites dog**.

To fix this,
To fix this, the position (scalar i) passes through 12,288 sinusoidal functions of varying frequencies.

This positional vector is simply added to the embedding vector.



Positional Matrix
(2048 x 12288)



Sequence
Embedding Matrix
(2048 x 12288)

Positional
Embedding Matrix
(2048 x 12288)

Dimensional
Tracker

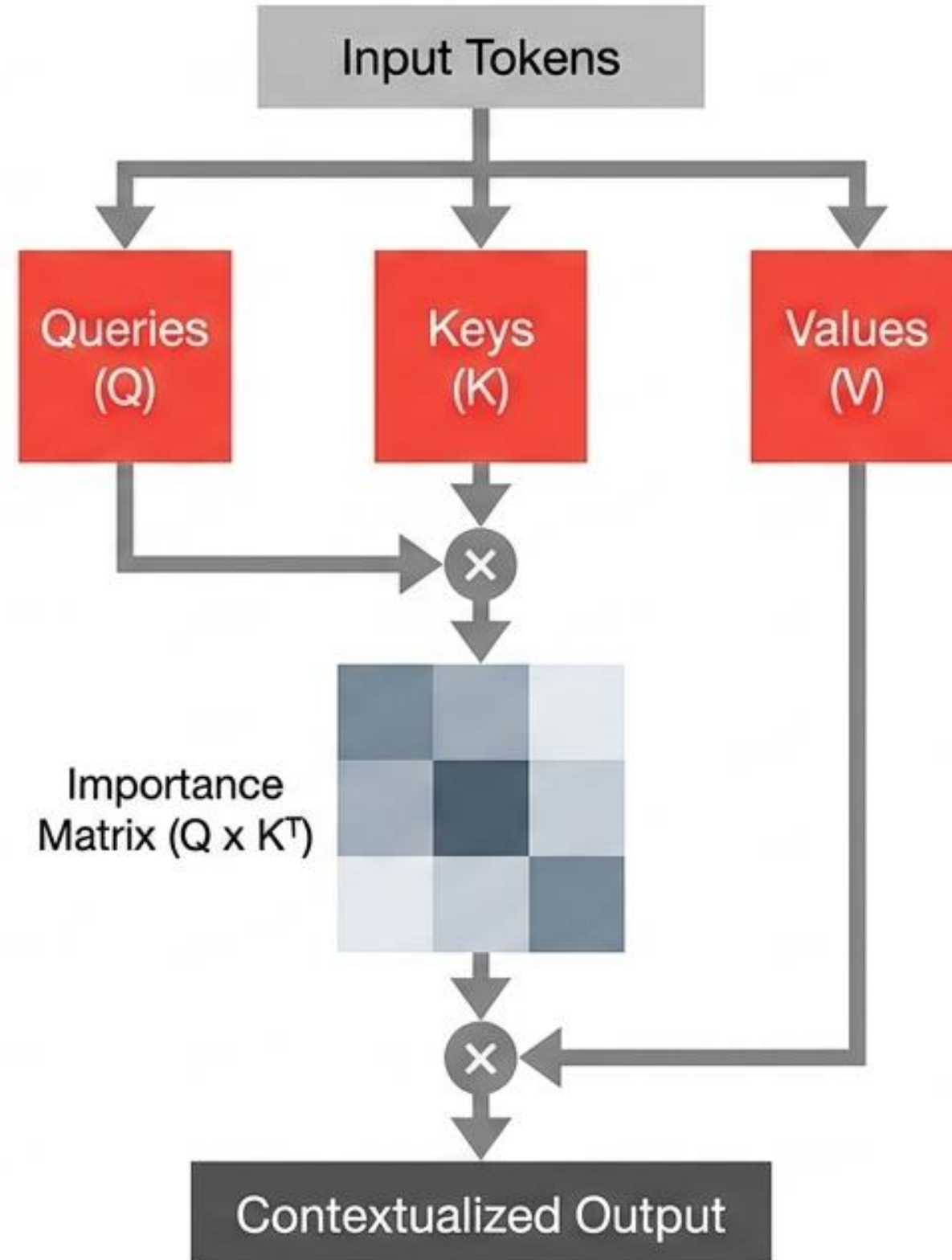
Current Matrix
Shape Remains:
2048 x 12288

Attention: The only operation where words interact.

Attention Mechanism:

The model predicts which input tokens to focus on. Queries (Q) and Keys (K) multiply to score the relevance of every token to every other token.

The result is normalized via softmax to create an Importance Matrix, which dictates how much of the Values (V) to mix together.



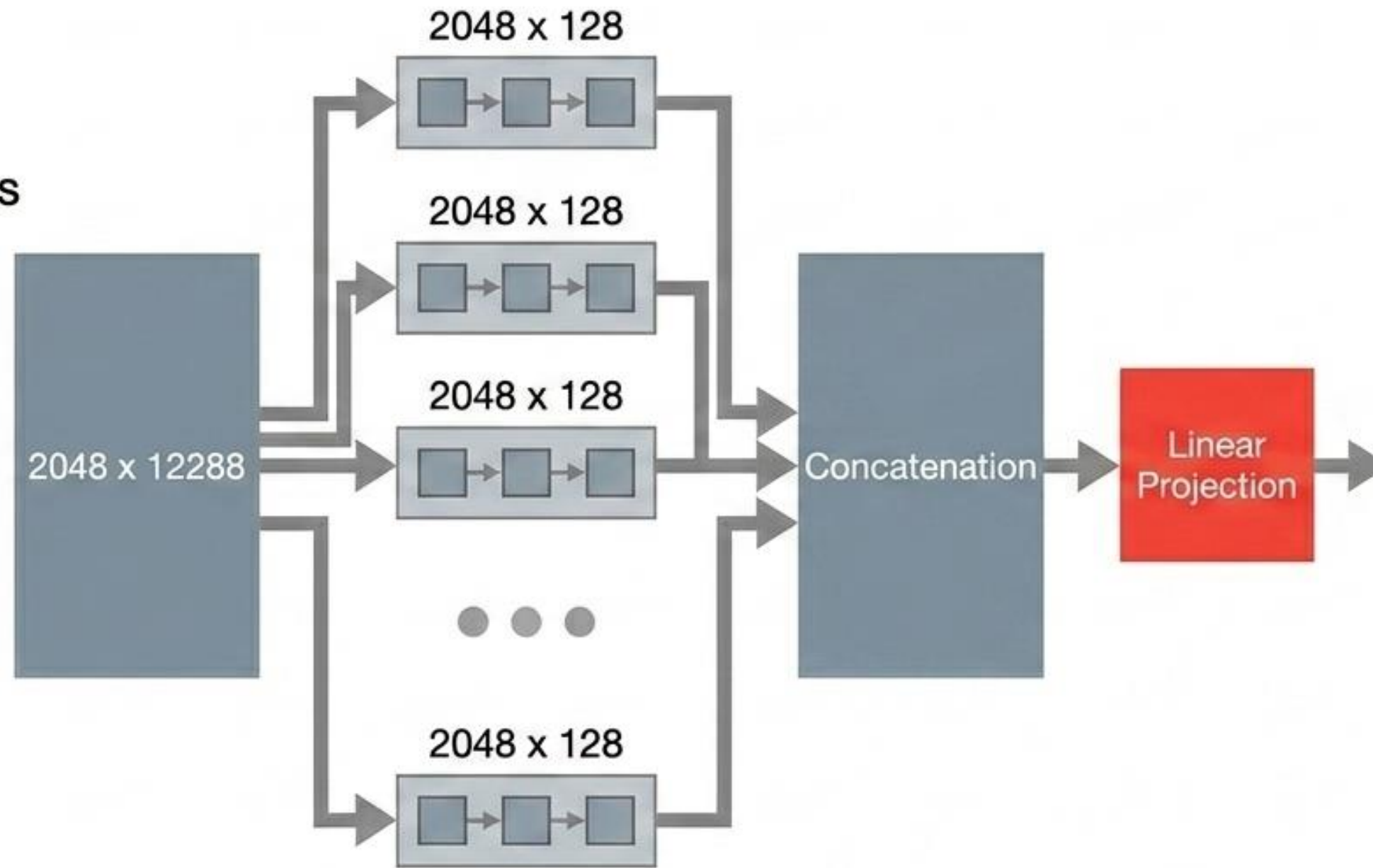
Dimensional
Tracker

Operating Shape:
2048 x 12288

96 unique perspectives processed in parallel.

GPT-3 uses Multi-Head Attention. The sequence is divided into 96 separate heads.

This allows the model to simultaneously learn 96 different types of relationships (e.g., grammatical structure, tone, entity matching) across the same text.



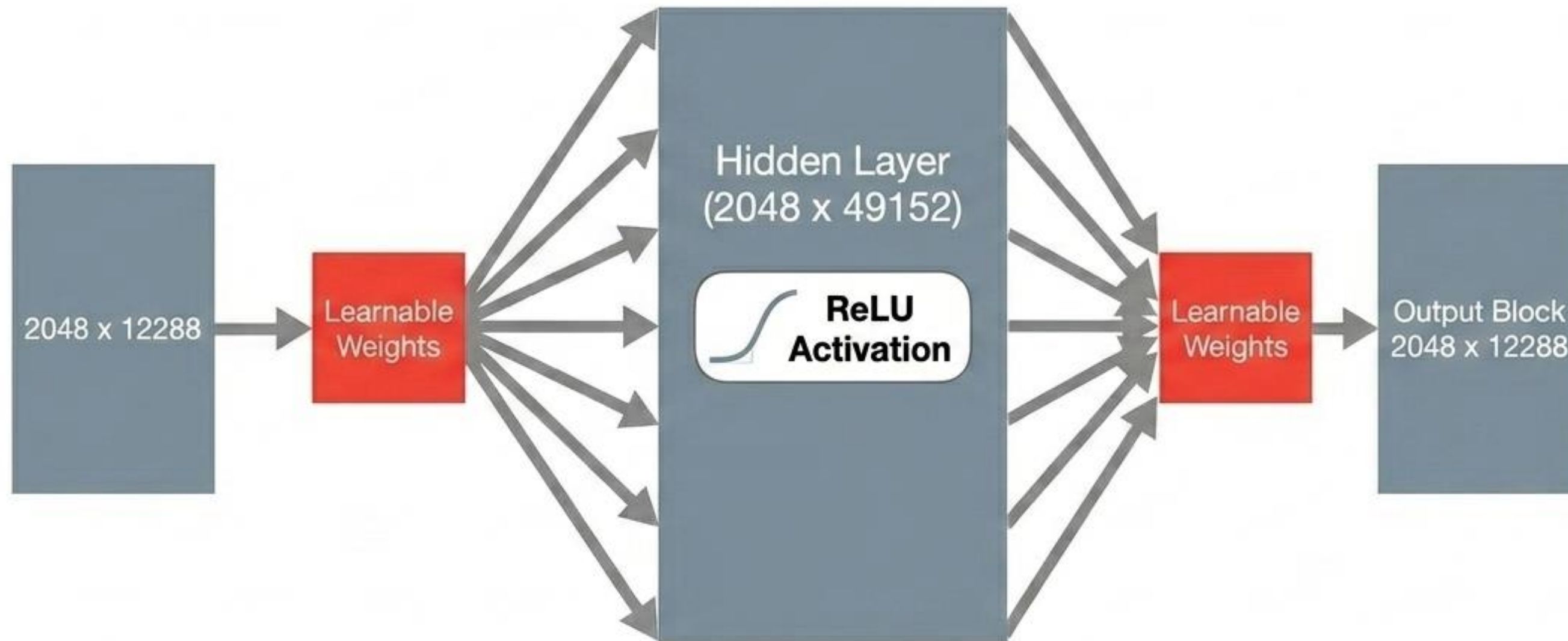
Dimensional
Tracker

Split:
 $96 \times (2048 \times 128)$

Concatenation:
 2048×12288

Linear
Projection

Synthesizing context through the Feed Forward layer.



A classic Multi-Layer Perceptron (MLP) with one hidden layer.

The hidden layer expands the data to exactly 4x its size to process the complex context gathered during the Attention phase.

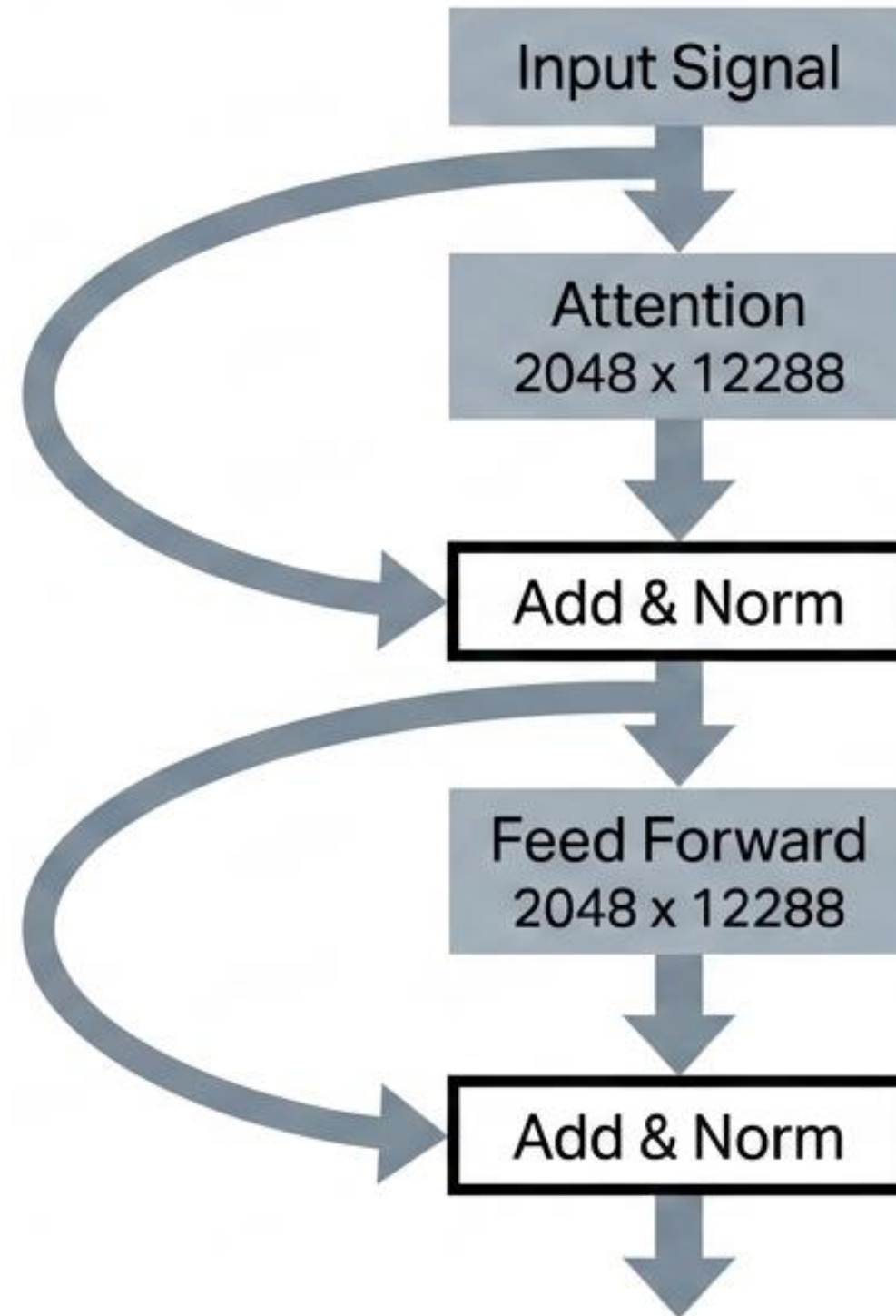
Unlike Attention, this block operates on each sequence position entirely independently.

Dimensional Tracker

Expansion:
2048 x 49152

Compression:
2048 x 12288

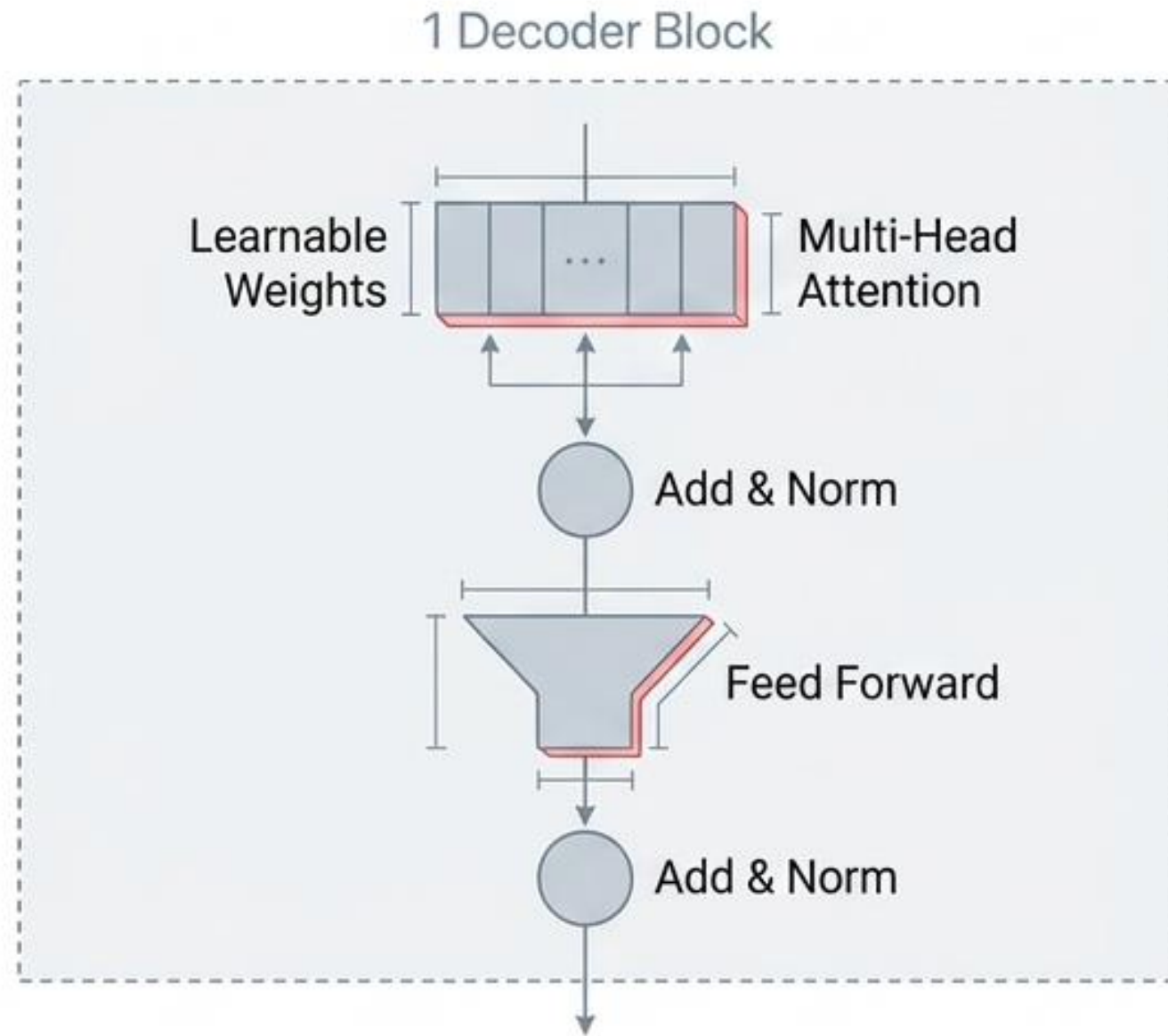
Stabilizing the signal with Add & Norm.



Deep networks suffer from signal degradation.

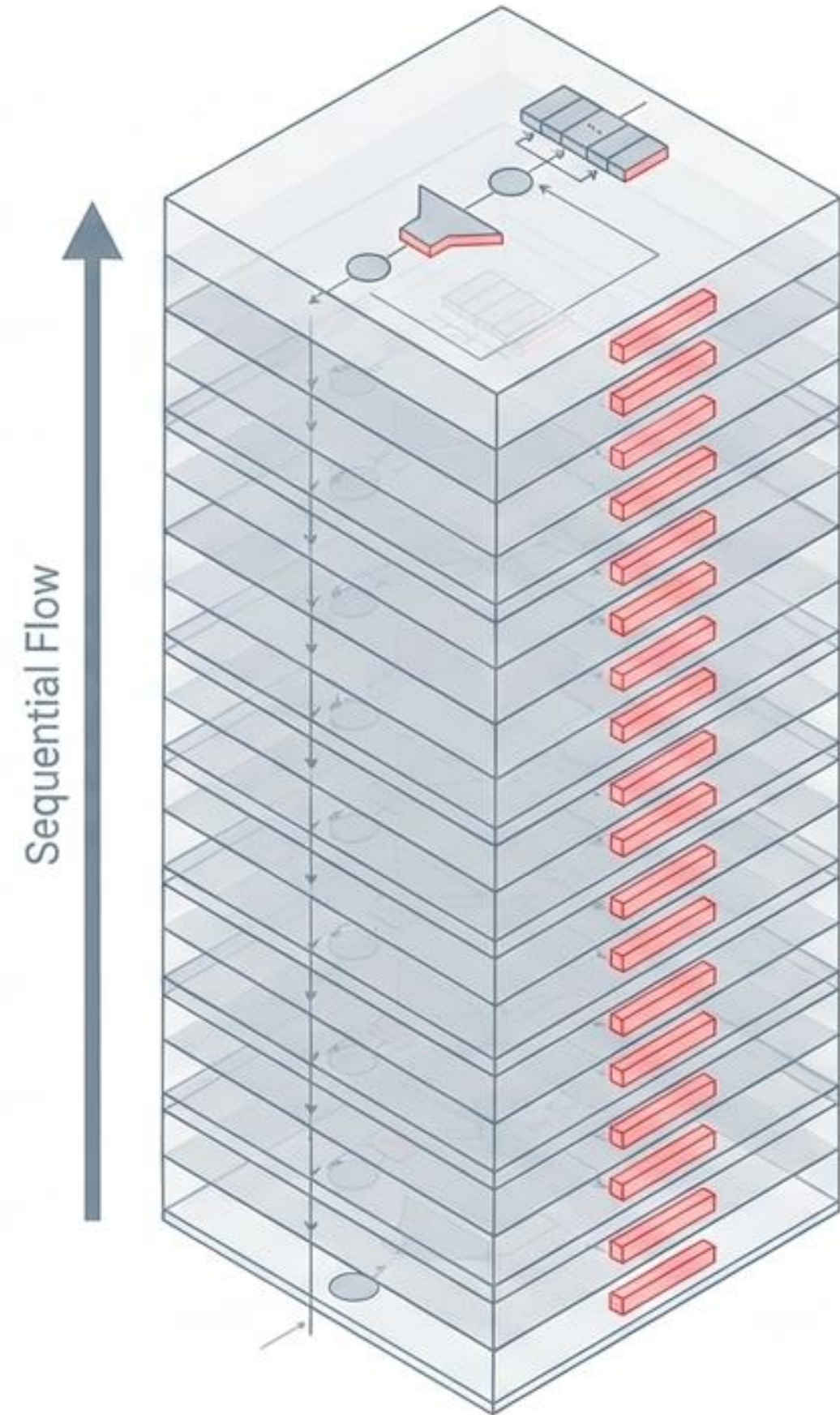
By adding the input directly to the output (a technique adopted from ResNet) and applying layer normalization, GPT-3 ensures vital information survives the journey through the machine.

The sheer depth of the machine.

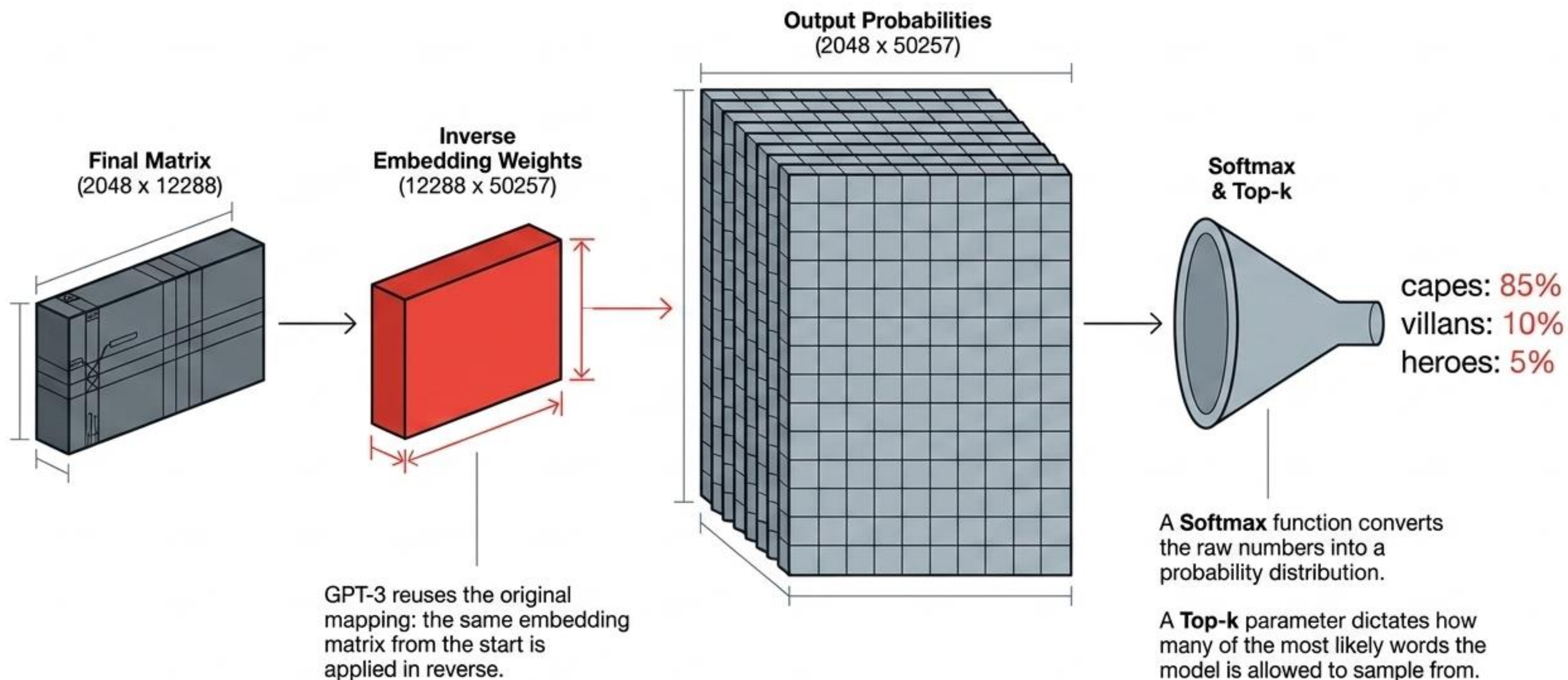


The input matrix is processed sequentially through 96 identical layers.

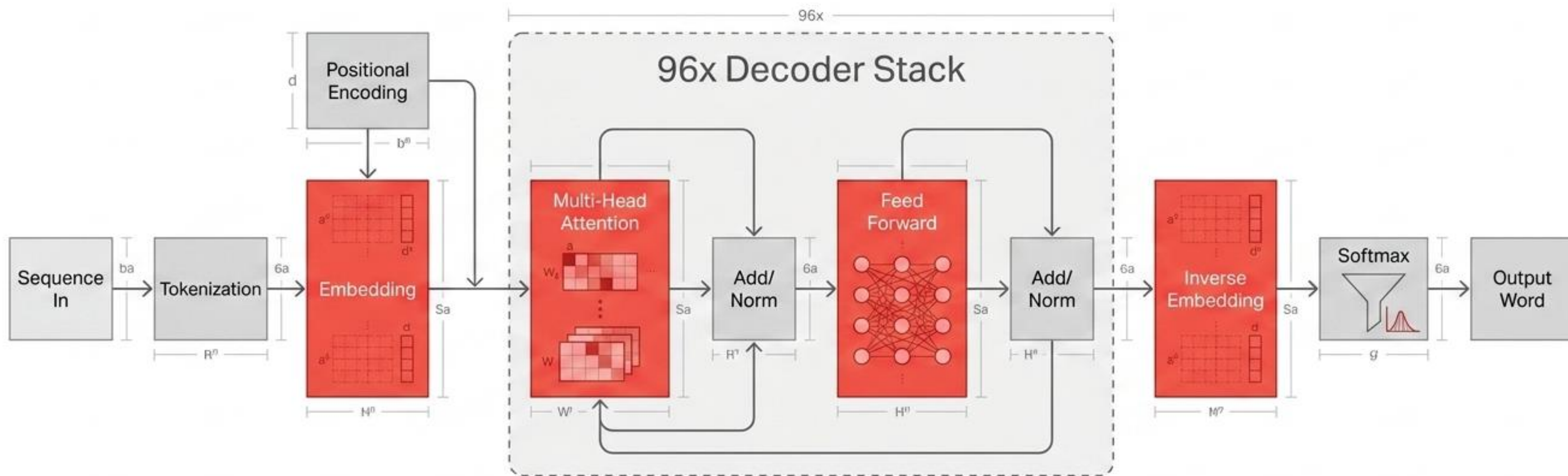
While the architecture of each layer is identical, every single layer contains its own completely unique, independent set of learnable weights.



Decoding the matrix back into human language.



The GPT-3 Architecture Blueprint



A few matrix multiplications, some linear algebra, and 175 billion learned parameters working in perfect sequence.

The Dimensional Journey of a Token

Pipeline Step	Resulting Matrix Shape
Raw Input Sequence	2048 x 1
One-Hot Encoded	2048 x 50257
Dense Embedded	2048 x 12288
Single Attention Head	2048 x 128
Feed-Forward Hidden Layer	2048 x 49152
Final Output Probabilities	2048 x 50257

The physical shape of data as it stretches and compresses through the model.